

Jalon 5 — Rapport de Développement, Sécurité & Tests (Version Bêta)

Projet : OptiFleet — Plateforme de gestion de flotte automobile **Auteur :** Kahil Mokhtari **Formation :** CDA — Concepteur Développeur d'Applications **Échéance :** 29/05/2026 **Dépôt Git :** <https://github.com/Hakkai19z/optifleet-app> **Tag de référence :** `v1.0.0` (commit `d01a8e1`) **Branches :** `main` (stable), `develop` (intégration continue)

Table des matières

- 1. [Code source de l'application \(version bêta\)](#).
- 2. [Intégration de l'API externe \(Google Maps\)](#).
- 3. [Intégration Continue \(CI\)](#).
- 4. [Politique de tests \(Chapitre X\)](#).
- 5. [Analyse de sécurité & conformité \(Chapitre IX\)](#).
- 6. [Bilan d'avancement](#)

1. Code source de l'application (version bêta)

1.1 Architecture livrée

L'application OptiFleet est une **Single Page Application (React)** consommant une **API REST Symfony 7**, conformément au socle technique imposé.

Couche	Technologie	Rôle
Front-end	React 18 + Vite + Tailwind CSS 3	Interface utilisateur (SPA)
Back-end	Symfony 7 + API Platform 3	API REST + logique métier
Base de données	PostgreSQL 16	Persistance relationnelle
ORM	Doctrine	Mapping objet-relationnel
Authentification	LexikJWTAuthenticationBundle (RS256)	Jetons JWT
Conteneurisation	Docker + Docker Compose	Environnement reproductible

1.2 Fonctionnalités implémentées

Gestion des véhicules (CRUD complet)

- Création, lecture, modification, suppression de véhicules
- Champs : immatriculation (format AA-000-AA validé), marque, modèle, année, kilométrage, statut, catégorie, adresse, quota kilométrique annuel
- Géolocalisation GPS automatique via l'adresse

Système de rôles et contrôle d'accès

- Trois rôles hiérarchisés : **ADMIN** > **GESTIONNAIRE** > **CONDUCTEUR**
- Chaque compte voit uniquement les fonctionnalités correspondant à son rôle

Affectation des véhicules (cœur métier)

- Le **gestionnaire** affecte les véhicules aux conducteurs
- Vue d'ensemble de la flotte en temps réel (qui conduit quoi)
- Libération / changement de véhicule avec mise à jour automatique des statuts

Espace conducteur

- Le conducteur consulte uniquement son véhicule affecté, son quota kilométrique, sa localisation et son historique d'entretien
- Signalement de problème (génère une alerte au gestionnaire)

Quotas kilométriques

- Suivi du kilométrage par rapport au quota annuel
- Barres de progression colorées (vert / orange / rouge) et alertes de dépassement

Maintenance & alertes

- Suivi des entretiens (révision, vidange, contrôle technique...)
- Génération automatique d'alertes (échéances dépassées)

Tableau de bord

- KPIs (véhicules par statut, taux de disponibilité, coûts de maintenance)
- Graphiques (répartition par statut, évolution des coûts)
- Section quotas kilométriques en alerte

1.3 Déploiement local

Le fichier `docker-compose.yml` orchestre trois services. Lancement en une commande :

```
docker compose up -d
```

Service	URL	Description
Front-end	http://localhost:3000	Application React
Back-end (API)	http://localhost:8000/api	API REST Symfony
Base de données	localhost:5432	PostgreSQL 16

Initialisation des données de démonstration :

```
docker compose exec app php bin/console doctrine:migrations:migrate --no-interaction
docker compose exec app php bin/console doctrine:fixtures:load --no-interaction
```

Comptes de test :

Rôle	Email	Mot de passe
Administrateur	admin@optifleet.fr	Admin@1234
Gestionnaire	gestionnaire@optifleet.fr	Gest@1234
Conducteur	conducteur@optifleet.fr	Cond@1234

2. Intégration de l'API externe (Google Maps)

L'application intègre l'**API Google Maps Geocoding** pour géolocaliser automatiquement les véhicules à partir de leur adresse.

2.1 Fonctionnement

- À chaque création/modification d'un véhicule avec une adresse, un **listener Doctrine** (`VehiculeGeocodingListener`) déclenche un appel à l'API Geocoding.
- L'API retourne les coordonnées (latitude/longitude) qui sont persistées avec le véhicule.
- La carte est affichée sur la fiche véhicule (embed Google Maps) et dans l'espace conducteur.

2.2 Sécurité de la clé d'API

La clé Google Maps est stockée dans une **variable d'environnement** (`GOOGLE_MAPS_API_KEY` dans le fichier `.env` , non versionné), **jamais codée en dur** dans le code source.

2.3 Test réel effectué

Adresse saisie	Coordonnées retournées
Tour Eiffel, Paris	lat: 48.85837, lng: 2.29448
1 Place Bellecour, Lyon	lat: 45.75894, lng: 4.83096
20 Rue de la République, Marseille	lat: 43.29787, lng: 5.37310

Statut :  Opérationnel et testé.

3. Intégration Continue (CI)

Une pipeline **GitHub Actions** (`.github/workflows/ci.yml`) est en place. Elle se déclenche automatiquement à chaque `push` sur `main` / `develop` et à chaque `pull_request` vers `main` .

3.1 Job Backend

Étape	Outil	Description
Setup PHP 8.3	shivammathur/setup-php	Avec extensions pdo_pgsql, intl, mbstring + xdebug
Service PostgreSQL 16	Docker service	Base de données de test isolée
Install dépendances	Composer	<code>composer install</code>
Analyse de style	php-cs-fixer	Vérification PSR (dry-run)
Analyse statique	PHPStan (niveau 5)	Détection d'erreurs sans exécution
Génération clés JWT	OpenSSL	Clés RSA 4096 bits
Migrations	Doctrine	Création du schéma
Chargement fixtures	Doctrine Fixtures	Données de test
Tests	PHPUnit 11	Tests unitaires + fonctionnels avec couverture

3.2 Job Frontend

Étape	Outil	Description
Setup Node.js 20	actions/setup-node	Avec cache npm
Install dépendances	npm ci	Installation déterministe
Lint	ESLint	Qualité du code
Tests	Vitest	Tests avec couverture
Build	Vite	Vérification de la compilation production

La pipeline vérifie donc **trois niveaux de qualité** : style de code, analyse statique, tests automatisés, plus la compilation du front. Chaque commit est validé automatiquement, conformément à la démarche CI/CD.

4. Politique de tests (Chapitre X)

4.1 Outils utilisés

Périmètre	Framework	Type de tests
Back-end	PHPUnit 11	Tests unitaires (services métier)
Back-end	PHPUnit + WebTestCase	Tests fonctionnels (endpoints HTTP)
Front-end	Vitest + Testing Library	Tests de composants et hooks React
Analyse statique	PHPStan niveau 5	Vérification de typage et cohérence
Style de code	php-cs-fixer (PSR)	Conventions de codage

4.2 Couverture des tests unitaires (back-end)

Les tests unitaires couvrent les **services métier**, cœur de la logique applicative :

Fichier de test	Classe testée	Points vérifiés
VehiculeServiceTest	VehiculeService	Validation d'immatriculation, calcul de disponibilité, statistiques par statut
AlerteServiceTest	AlerteService	Création d'alertes, vérification des échéances, comptage des alertes actives
EntretienServiceTest	EntretienService	Planification, détection d'échéance, calcul du coût total par période

Exemple de cas de test unitaire pertinent (VehiculeServiceTest) :

On vérifie que la méthode de validation d'immatriculation `validerImmatriculation()` retourne `true` pour le format légal français `AB-123-CD` et `false` pour une saisie invalide (`ABC-12-D` , chaîne vide, etc.). Ceci garantit qu'aucun véhicule ne peut être enregistré avec une plaque mal formée.

4.3 Tests fonctionnels (back-end)

Les tests fonctionnels simulent de vraies requêtes HTTP sur l'API via `WebTestCase` :

Test	Scénario	Résultat attendu
testLoginSuccess	Connexion avec identifiants admin valides	200 + jeton JWT retourné
testLoginEchec	Connexion avec mauvais mot de passe	401 Unauthorized
testAccessProtectedRouteWithoutToken	Accès à <code>/api/vehicules</code> sans jeton	401 Unauthorized

Ces tests valident l'authentification JWT de bout en bout (login → obtention du token) et le contrôle d'accès sur les routes protégées.

4.4 Tests front-end (React)

Fichier de test	Composant / Hook	Points vérifiés
<code>Badge.test.jsx</code>	Badge	Rendu des libellés et classes CSS par statut
<code>Button.test.jsx</code>	Button	Variantes, état désactivé, gestion du clic
<code>VehiculeCard.test.jsx</code>	VehiculeCard	Affichage des données véhicule
<code>useAuth.test.js</code>	Hook useAuth	Logique d'authentification, vérification des rôles

4.5 Tests d'intégration

- **Base de données réelle** : les tests fonctionnels s'exécutent contre une base PostgreSQL `optifleet_test` dédiée (isolée de la base de développement), avec migrations et fixtures chargées.
- **API externe réelle** : l'intégration Google Maps Geocoding a été testée avec de vraies adresses (cf. §2.3), validant la consommation du service web externe et le traitement des réponses.

4.6 Résultats actuels

```
=== BACK-END (PHPUnit 11) ===
OK (15 tests, 35 assertions) → 100 % de réussite

=== FRONT-END (Vitest) ===
Test Files 4 passed (4)
Tests 19 passed (19) → 100 % de réussite

=== ANALYSE STATIQUE (PHPStan niveau 5) ===
[OK] No errors

=== TOTAL : 34 tests automatisés, tous au vert ✅ ===
```

Note sur la couverture chiffrée : la métrique de couverture (%) est générée en CI via Xdebug (`--coverage-clover`). En environnement de développement local, le driver de couverture n'est pas installé pour préserver les performances. La couverture est concentrée sur les **3 services métier critiques** (validation, alertes, entretiens), qui représentent le cœur de la logique applicative.

5. Analyse de sécurité & conformité (Chapitre IX)

L'application respecte les bonnes pratiques de sécurité web et adresse les principales failles du **Top 10 OWASP**.

5.1 Injection SQL

Mesure : utilisation **exclusive de Doctrine ORM**. Toutes les requêtes passent par le QueryBuilder de Doctrine avec des **paramètres liés** (`setParameter()`). Aucune concaténation de chaîne SQL avec des entrées utilisateur.

```
// Exemple : requête paramétrée (GestionnaireController)
->where('u.role = :role')
->setParameter('role', 'CONDUCTEUR')
```

Les entrées utilisateur ne sont jamais insérées directement dans une requête. **Risque d'injection SQL éliminé.**

5.2 XSS (Cross-Site Scripting)

Mesure : le front-end React **échappe automatiquement** toutes les variables affichées dans le JSX (protection native de React). L'API communique exclusivement en **JSON** (pas de rendu HTML côté serveur), ce qui supprime la surface d'attaque XSS classique. Aucune utilisation de `dangerouslySetInnerHTML` .

5.3 CSRF

Mesure : l'API est **stateless** et authentifiée par **jeton JWT** transmis dans l'en-tête `Authorization: Bearer` . Aucune session par cookie n'est utilisée pour l'authentification de l'API, ce qui rend les attaques CSRF inopérantes (un site tiers ne peut pas accéder au jeton stocké côté client). C'est l'approche recommandée pour une API REST consommée par une SPA.

5.4 Authentification et mots de passe

Mesure	Implémentation
Hachage	Bcrypt avec coût 12 (<code>security.yaml</code>)
Stockage	Aucun mot de passe en clair, jamais
Jetons	JWT signés en RS256 (clés RSA 4096 bits)
Durée de vie	Token d'accès : 15 minutes (900 s)
Validation email	Contrainte <code>Assert\Email</code> sur l'entité

5.5 Protection contre le brute force

Mesure : un **Rate Limiter Symfony** est branché sur l'endpoint de connexion :

- Politique : **fenêtre glissante (sliding window)**
- Limite : **5 tentatives par tranche de 15 minutes**, par adresse IP
- Au-delà : réponse **HTTP 429 (Too Many Requests)** avec délai d'attente indiqué
- Le compteur est réinitialisé après une connexion réussie

Test effectué : après 5 échecs consécutifs, la 6^e tentative est bloquée avec un code 429. ✓

5.6 Données personnelles & RGPD ✓

Mesure : conformément au RGPD, l'utilisateur peut **supprimer son compte** et donc ses données personnelles via l'endpoint `DELETE /api/auth/delete-account`. Les mots de passe sont hachés (Bcrypt) et jamais exposés dans les réponses API. En production, les communications sont prévues en HTTPS (TLS).

5.7 Contrôle d'accès (rôles et permissions) ✓

Mesure : hiérarchie de rôles Symfony stricte :

```
role_hierarchy:
    ROLE_ADMIN:          [ROLE_GESTIONNAIRE]
    ROLE_GESTIONNAIRE:   [ROLE_CONDUCTEUR]
    ROLE_CONDUCTEUR:     [ROLE_USER]
```

- Chaque endpoint sensible est protégé par l'attribut `#[IsGranted(...)]` ou la propriété `security` d'API Platform.
- Un conducteur ne peut **pas** accéder aux endpoints de gestion (affectation, CRUD véhicules, administration).
- Les opérations de gestion de flotte exigent `ROLE_GESTIONNAIRE` minimum ; l'administration des utilisateurs exige `ROLE_ADMIN`.

Test fonctionnel : l'accès à une route protégée sans jeton retourne bien 401 (cf. §4.3).

5.8 Validation des données en entrée ✓

Mesure : contraintes de validation Symfony sur les entités :

- Immatriculation : expression régulière `^[A-Z]{2}-[0-9]{3}-[A-Z]{2}$`
- Email : `Assert\Email`
- Champs obligatoires : `Assert\NotBlank`
- Longueurs : `Assert\Length`
- Quota / kilométrage : `Assert\Positive` / `Assert\PositiveOrZero`
- Statut : `Assert\Choice` (liste blanche de valeurs)

5.9 CORS ✓

Mesure : `NelmioCorsBundle` configuré. En production, l'origine autorisée est restreinte via la variable d'environnement `CORS_ALLOW_ORIGIN` (expression régulière limitant aux domaines légitimes).

5.10 Synthèse OWASP

Faible OWASP	Statut	Mesure
Injection SQL	✓ Couvert	Doctrine ORM + requêtes paramétrées
XSS	✓ Couvert	Échappement React + API JSON
CSRF	✓ Couvert	API stateless JWT (pas de cookie de session)
Authentification faible	✓ Couvert	Bcrypt coût 12 + JWT RS256
Brute force	✓ Couvert	Rate Limiter 5/15 min
Contrôle d'accès défaillant	✓ Couvert	Hiérarchie de rôles + Voters
Données personnelles	✓ Couvert	RGPD : suppression de compte
Entrées non validées	✓ Couvert	Contraintes Assert Symfony

6. Bilan d'avancement

6.1 Fonctionnalités terminées (✓)

- CRUD complet des véhicules avec validation
- Authentification JWT (RS256) + protection brute force
- Système de rôles cohérent (Admin / Gestionnaire / Conducteur)
- Affectation des véhicules aux conducteurs (cœur métier)
- Vue flotte temps réel + espace conducteur dédié
- Quotas kilométriques avec alertes visuelles
- Gestion des entretiens et alertes automatiques
- Tableau de bord avec KPIs et graphiques
- Intégration Google Maps (géolocalisation) opérationnelle
- Signalement de problème par le conducteur
- Conteneurisation Docker (3 services)
- Pipeline CI/CD (GitHub Actions) : style + analyse statique + tests + build
- 34 tests automatisés (100 % au vert)
- Conformité RGPD (suppression de compte)

6.2 Points à finaliser en juin (→ SOON)

Point	Plan d'action
Couverture de code chiffrée	Activer le rapport de couverture en CI et viser ≥ 70 % sur les services métier
Headers HTTP de sécurité	Ajouter <code>nelmio/security-bundle</code> (X-Frame-Options, HSTS) pour la production
PHPStan niveau 6	Compléter le typage générique des collections (<code>array<...></code>) pour monter d'un niveau
Tests end-to-end front	Ajouter quelques scénarios de parcours utilisateur complets (Cypress/Playwright)
Déploiement continu (CD)	Finaliser la publication d'image Docker + procédure de mise en production

6.3 Positionnement par rapport au planning

Le projet est **conforme au planning prévisionnel**. La version bêta livrée à ce jalon est **fonctionnelle dans sa quasi-totalité** : toutes les fonctionnalités principales sont implémentées et testées, l'application est déployable en une commande Docker et utilisable pour l'ensemble des cas d'usage prévus. Le mois de juin sera consacré à la finalisation qualité (couverture, durcissement sécurité production) et au volet déploiement (CD).